

LAPORAN PRAKTIKUM 1 PEMOGRAMAN BERORIENTASI OBJEK
CLASS, OBJECT, CONSTRUCTOR dan METHOD

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah Pemograman Berorientasi Objek yang membahas tentang konsep Class, Object, Constructor, dan Method dalam Java Eclipse.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II.....	2
PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Langkah kerja	2
2.2.1 Buka eclipse.....	2
2.2.2 Membuat Java Project.....	3
2.2.3 Class 1 “Rekening”	3
2.2.4 Class 2 “Main”	6
2.3 Tugas Praktikum.....	10
BAB III	14
KESIMPULAN	14
DAFTAR PUSTAKA	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek merupakan salah satu paradigma penting dalam dunia pemrograman yang menekankan pada pemodelan perangkat lunak berdasarkan entitas dunia nyata. Konsep ini menghadirkan cara berpikir baru dalam membangun aplikasi, di mana Class berfungsi sebagai cetak biru, Object menjadi representasi nyata dari class, Constructor digunakan untuk menginisialisasi state awal, dan Method berperan dalam mendefinisikan perilaku objek. Dengan pendekatan ini, program tidak hanya lebih terstruktur, tetapi juga lebih mudah dipelihara dan dikembangkan. Praktikum PBO dengan tema Class, Object, Constructor, dan Method memberikan kesempatan bagi mahasiswa untuk memahami bagaimana interaksi antar objek dapat memanipulasi state serta membentuk sistem yang dinamis, seperti simulasi mini bank yang mencontohkan pengelolaan rekening, saldo, dan transaksi.

1.2 Tujuan

1. Memahami peran Class sebagai cetak biru dari objek.
2. Mengetahui cara kerja Constructor dalam menginisialisasi state awal objek.
3. Mempelajari penggunaan Method untuk memanipulasi state dan perilaku objek.

1.3 Manfaat

1. Memberikan pemahaman praktis tentang bagaimana objek berinteraksi dalam program.
2. Melatih keterampilan mahasiswa dalam mengimplementasikan konsep PBO di Java.
3. Menjadi bekal dasar untuk pengembangan aplikasi yang lebih kompleks dan efisien.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dibangun dengan pendekatan yang meniru entitas dunia nyata. Empat konsep utama yang menjadi fondasi adalah Class, Object, Constructor, dan Method.

- Class adalah cetak biru (blueprint) yang mendefinisikan struktur data berupa atribut (state) dan perilaku (method). Class tidak menyimpan data secara langsung, melainkan berfungsi sebagai tipe referensi yang digunakan untuk membuat objek.
- Object merupakan instansiasi nyata dari sebuah class yang dialokasikan ke dalam memori. Setiap objek memiliki state yang independen, sehingga perubahan pada satu objek tidak memengaruhi objek lain meskipun berasal dari class yang sama.
- Constructor adalah blok kode khusus yang dijalankan otomatis saat objek dibuat menggunakan keyword new. Fungsinya untuk menginisialisasi nilai awal atribut agar objek siap digunakan.
- Method adalah kumpulan instruksi yang mendefinisikan perilaku atau aksi dari objek. Method digunakan untuk memanipulasi state, seperti menambah saldo, menampilkan informasi, atau melakukan operasi lain sesuai kebutuhan program.

2.2 Langkah kerja

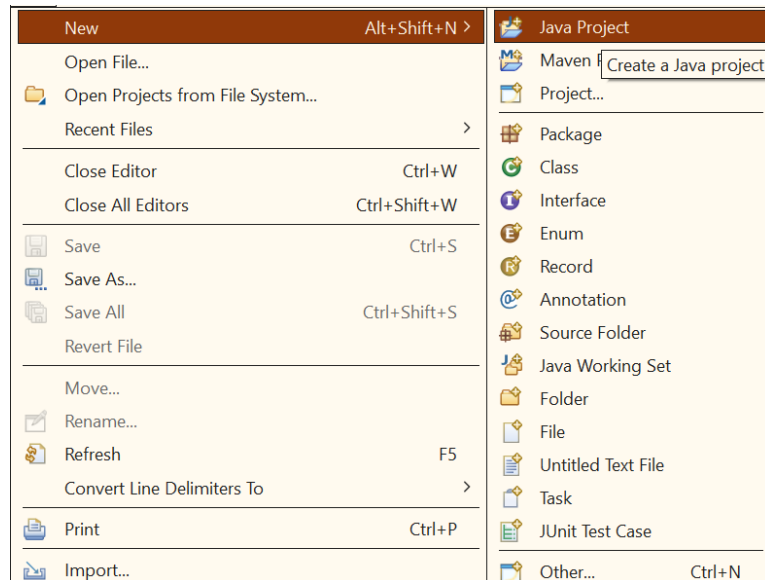
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

2.2.2 Membuat Java Project

1. Klik file
2. Klik java project

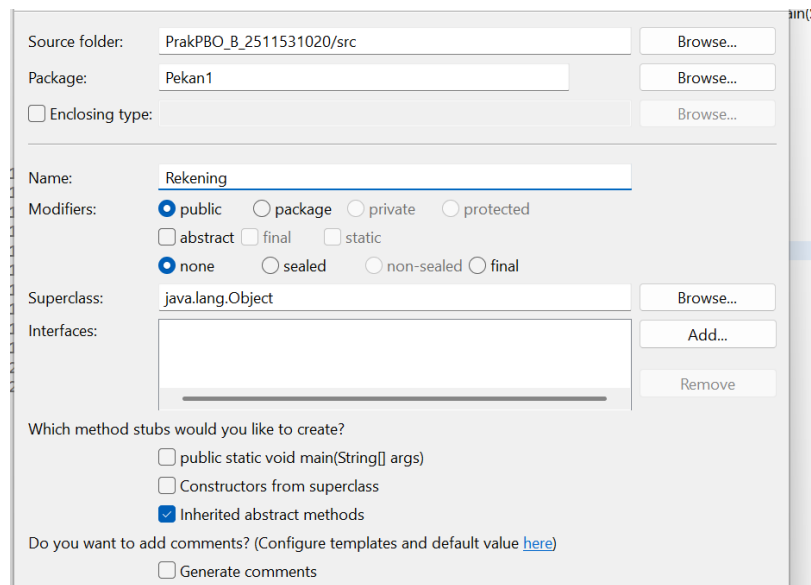


Gambar 2.2 Java Project

3. Buat nama project: PrakPBO_B_2511531020

2.2.3 Class 1 “Rekening”

1. Membuat package pekan1 dan class “Rekening”



Gambar 2.3 Membuat Nama package dan class Rekening

2. Klik Finish

3. Kode Program

```

4. package Pekan1;
5.
6. public class Rekening {
7.     String nomorRekening;
8.     String namaPemilik;
9.     double saldo;
10.
11.     public Rekening(String nomor, String nama, double saldoAwal) {
12.         nomorRekening = nomor;
13.         namaPemilik = nama;
14.         saldo = saldoAwal;
15.         System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo RP" + saldo);
16.     }
17.     public void setorTunai(double nominal) {
18.         if(nominal > 0) {
19.             saldo += nominal;
20.             System.out.println("Setor tunai RP" + nominal +
"berhasil. Saldo saat ini8 :: Rp" + saldo);
21.         }else {
22.             System.out.println("Gagal: nominal setor harus dari
0!");
23.         }
24.     }
25.
26.     public void tarikTunai(double nominal) {
27.         if (nominal < 10000) {
28.             // Aturan 1: Minimal penarikan 10.000

```

```

29.         System.out.println("Transaksi Gagal: Minimal nominal
           penarikan 10.000");
30.     } else if (nominal > saldo) {
31.         // Aturan 2: Saldo tidak boleh minus / tidak mencukupi
32.         System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
           Saldo Anda: Rp" + saldo);
33.     } else {
34.         // Aturan 3: Jika lolos semua aturan, kurangi saldo
35.         saldo -= nominal;
36.         System.out.println("Tarik tunai Rp" + nominal + " berhasil.
           Saldo saat ini: Rp" + saldo);
37.     }
38. }
39. public void cekInformasi() {
40.     System.out.println("--- INFO REKENING ---");
41.     System.out.println("No. Rekening : " + nomorRekening);
42.     System.out.println("Nama pemilik : " + namaPemilik);
43.     System.out.println("Saldo Akhir : Rp " + saldo);
44.     System.out.println("-----");
45. }
46.
47.
48.
49. }

```

Gambar 2.4 Class 1 “Rekening”

4. Analisis Langkah

a. Struktur Program

Kelas Rekening berada dalam package Pekan1.

Kelas ini berfungsi sebagai model utama untuk simulasi rekening bank sederhana.

b. Elemen Sintaks Utama

- Constructor Rekening(String nomor, String nama, double saldoAwal) → digunakan untuk menginisialisasi objek rekening baru dengan nomor, nama pemilik, dan saldo awal.
- setorTunai(double nominal) → method untuk menambah saldo rekening dengan validasi nominal > 0.

- tarikTunai(double nominal) → method untuk mengurangi saldo rekening dengan aturan minimal penarikan Rp10.000 dan saldo tidak boleh minus.
- cekInformasi() → method untuk menampilkan informasi rekening (nomor, nama pemilik, saldo).
- Variabel penting:
 - nomorRekening → menyimpan nomor rekening.
 - namaPemilik → menyimpan nama pemilik rekening.
 - saldo → menyimpan jumlah saldo saat ini.

c. Alur Kerja Sintaks:

- Saat objek dibuat dengan constructor, sistem langsung menampilkan pesan rekening berhasil dibuat dengan saldo awal.
- Method setorTunai menambah saldo jika nominal valid, dan menampilkan pesan keberhasilan atau kegagalan.
- Method tarikTunai memeriksa aturan:
 - Jika nominal < 10.000 → transaksi gagal.
 - Jika nominal > saldo → transaksi gagal karena saldo tidak cukup.
 - Jika valid → saldo dikurangi dan pesan keberhasilan ditampilkan.
- Method cekInformasi menampilkan detail rekening ke konsol.

2.2.4 Class 2 “Main”

1. Membuat package pekan1 dan class Main

Source folder: PrakPBO_B_2511531020/src Browse...

Package: Pekan1 Browse...

☐ Enclosing type: Browse...

Name: Main

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.5 Membuat Nama package dan class Main

2. Klik Finish
3. Kode Program

```

PrakPBO_B_2511531020 > src > Pekan1 > Main > main(String[]) : void
1 package Pekan1;
2
3 import java.util.ArrayList;
4
5
6 public class Main {
7     public static void main (String [] args) {
8         Scanner input = new Scanner (System.in) ;
9         ArrayList<Rekening> daftarRekening = new ArrayList<>();
10        Rekening akunAktif = null; // Menunjuk ke akun yang sedang terpilih
11
12        boolean isRunning = true;
13
14        System.out.println("=== SISTEM PERBANKAN MINI ===");
15
16        while (isRunning) {
17            System.out.println("\nMenu Utama:");
18            System.out.println("1. Buka Rekening Baru");
19            System.out.println("2. Setor Tunai");
20            System.out.println("3. Tarik Tunai");
21            System.out.println("4. Cek Informasi Rekening");
22            System.out.println("0. Keluar");
23            System.out.println("Pilih menu: ");
24
25            int pilihan = input.nextInt();
26            input.nextLine(); //Membersihkan buffer enter
27
28            switch (pilihan) {
29                case 1:
30                    System.out.println("Masukkan No Rekening:");
31                    String no = input.nextLine();
32                    System.out.println("Masukkan Nama Pemilik:");
33                    String nama = input.nextLine();
34                    System.out.println("Masukkan Saldo Awal:");
35                    double saldo = input.nextDouble();

```

```

36
37
38 //Instansiasi Object / Menjalankan Constructor
39 Rekening barusakun = new Rekening(no, nama, saldo);
40 daftarRekening.add(barusakun); // Disimpan ke ArrayList
41 akunAktif = barusakun;
42 break;
43
44 case 2:
45     if (akunAktif == null) {
46         System.out.println("Error: Mohon maaf, Anda belum memiliki noor rekening!");
47     } else {
48         System.out.println("Masukkan nominal setor: ");
49         double setor = input.nextDouble();
50         akunAktif.setorTunai(setor); // Memanggil Behavior / method
51     }
52     break;
53
54 case 3:
55     if (akunAktif == null) {
56         System.out.println("Error: Mohon maaf, Anda belum memilih/memiliki rekening!");
57     } else {
58         System.out.print("Masukkan nominal tarik tunai: ");
59         double tarik = input.nextDouble();
60         akunAktif.tarikTunai(tarik);
61     }
62     break;
63
64 case 4:
65     if (akunAktif == null) {
66         System.out.println("Error: Anda belum membuka rekening!");
67     } else {
68         akunAktif.cekInformasi();
69     }
70     break;
71
72 case 5:
73     if (daftarRekening.isEmpty()) {
74         System.out.println("Belum ada rekening yang terdaftar di sistem.");
75     } else {
76         System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
77         String cariNo = input.nextLine();
78         boolean ditemukan = false;
79
80         for (Rekening r : daftarRekening) {
81             if (r.nomorRekening.equalsIgnoreCase(cariNo)) {
82                 akunAktif = r;
83                 ditemukan = true;
84                 System.out.println("Berhasil mengganti ke akun milik: " + r.namaPemilik);
85                 break;
86             }
87         }
88         if (!ditemukan) {
89             System.out.println("Error: Nomor rekening tidak ditemukan!");
90         }
91     }
92     break;
93
94 case 0:
95     isRunning = false;
96     System.out.println("Sistem ditutup. Terimakasih!");
97     break;
98
99 default:
100     System.out.println("Pilihan tidak Valid");
101 }
102 input.close();
103
104 }
105 }

```

Gambar 2.6 Class 2 “Main”

4. Analisis Langkah

a. Struktur Program

- Kelas Main berada dalam package Pekan1.

- Program ini berfungsi sebagai engine CLI (Command Line Interface) untuk simulasi sistem perbankan mini.

b. Elemen Sintaks Utama

- ArrayList<Rekening> daftarRekening → digunakan untuk menyimpan banyak objek rekening secara dinamis.
- Rekening akunAktif → menunjuk ke rekening yang sedang dipilih/aktif.
- Scanner input → menerima input dari user melalui konsol.
- switch-case → menangani pilihan menu utama.
- Method yang dipanggil dari class Rekening:
 - setorTunai() → menambah saldo.
 - tarikTunai() → mengurangi saldo dengan validasi.
 - cekInformasi() → menampilkan detail rekening.
- Variabel penting:
 - no, nama, saldo → data yang dimasukkan saat membuka rekening baru.
 - setor, tarik → nominal transaksi.
 - cariNo → nomor rekening yang dicari saat mengganti akun.
- Alur Kerja Sintaks
 - Program menampilkan menu utama dengan opsi buka rekening, setor tunai, tarik tunai, cek informasi, ganti akun, dan keluar.
 - Case 1: Membuka rekening baru → user memasukkan nomor, nama, saldo awal → objek Rekening dibuat dengan constructor.
 - Case 2: Setor tunai → jika akun aktif ada, saldo ditambah sesuai nominal.
 - Case 3: Tarik tunai → jika akun aktif ada, saldo dikurangi sesuai aturan validasi.
 - Case 4: Cek informasi → menampilkan detail rekening aktif.
 - Case 5: Ganti akun → mencari nomor rekening di daftarRekening, lalu menjadikannya akun aktif.
 - Case 0: Keluar dari sistem → loop berhenti dan program ditutup.
 - Default: Menangani input menu yang tidak valid.

5. Output

```
Console X
<terminated> Main [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
1
Masukkan No Rekening:
12345678
Masukkan Nama Pemilik:
Dinda Amelia
Masukkan Saldo Awal:
100000
Rekening atas nama Dinda Amelia berhasil dibuat dengan saldo Rp100000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
2
Masukkan nominal setor:
200000
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp300000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp250000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
4
--- INFO REKENING ---
No. Rekening : 12345678
Nama pemilik : Dinda Amelia
Saldo Akhir : Rp 250000.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
0
Sistem ditutup. Terimakasih!
```

Gambar 2.7 Output

2.3 Tugas Praktikum

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.

```
23 public void tarikTunai(double nominal) {
24     if (nominal < 10000) {
25         System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");
26     } else if (nominal > saldo) {
27         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
28     } else {
29         saldo -= nominal;
30         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
31     }
32 }
```

- public: Method ini bisa dipanggil dan diakses dari kelas lain (seperti dari Main.java).
 - void: Method ini hanya menjalankan proses dan tidak mengembalikan nilai (*return value*).
 - double nominal: Parameter input berjenis angka desimal untuk menampung jumlah uang yang ingin ditarik pengguna.
- Aturan: Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nomonal penarikan 10.000".

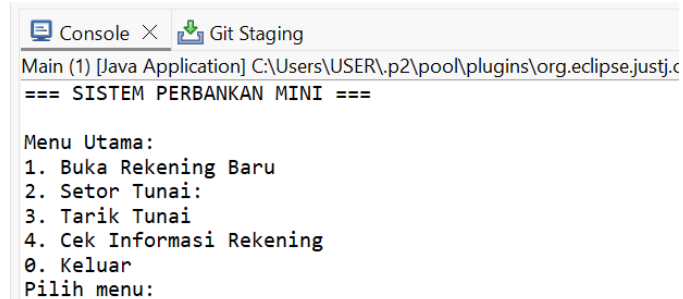
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 10000
Tarik tunai Rp10000.0 berhasil. Saldo saat ini: Rp40000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 5000
Transaksi Gagal: Minimal nominal penarikan 10.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 20000
Tarik tunai Rp20000.0 berhasil. Saldo saat ini: Rp20000.0
```

Kode method tarikTunai bertugas memvalidasi penarikan uang agar nominalnya minimal Rp10.000 dan tidak melebihi sisa saldo, di mana jika kedua syarat tersebut terpenuhi, saldo akan otomatis berkurang dan menampilkan sisa saldo terbaru ke layar. Hal ini terbukti pada tampilan output konsol yang menunjukkan transaksi berhasil saat menarik Rp10.000 dan Rp20.000 karena memenuhi syarat ketersediaan saldo, sedangkan penarikan Rp5.000 berhasil ditolak oleh sistem karena jumlahnya berada di bawah batas minimal Rp10.000.

- Terapkan method ini dengan menu nomor 3 pada class Main.



```

Console × Git Staging
Main (1) [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.c
=== SISTEM PERBANKAN MINI ===

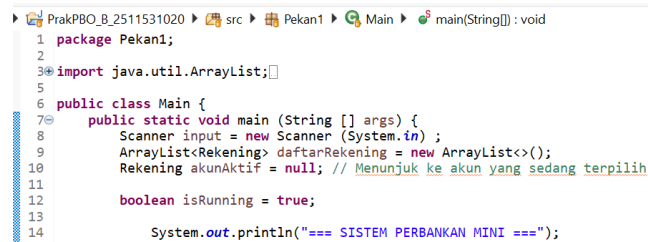
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai:
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:

```

Sudah berada di nomor 3

2. Tantangan Multi-Akun

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).



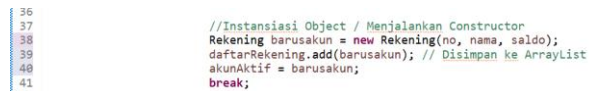
```

PrakPBO_B_2511531020 ▸ src ▸ Pekan1 ▸ Main ▸ main(String[]) : void
1 package Pekan1;
2
3 import java.util.ArrayList;
4
5
6 public class Main {
7     public static void main (String [] args) {
8         Scanner input = new Scanner (System.in) ;
9         ArrayList<Rekening> daftarRekening = new ArrayList<>();
10        Rekening akunAktif = null; // Menunjuk ke akun yang sedang terpilih
11
12        boolean isRunning = true;
13
14        System.out.println("=== SISTEM PERBANKAN MINI ===");

```

fungsinya sebagai wadah atau tempat penyimpanan dinamis untuk menampung banyak objek Rekening sekaligus.

- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.



```

36
37 //Instansiasi Object / Menjalankan Constructor
38 Rekening barusakun = new Rekening(no, nama, saldo);
39 daftarRekening.add(barusakun); // Disimpan ke ArrayList
40 akunAktif = barusakun;
41 break;

```

- Rekening barusakun = new Rekening(no, nama, saldo); (Baris 38) Membuat (*instansiasi*) objek rekening baru berdasarkan data nomor, nama, dan saldo awal yang diinputkan pengguna.
- daftarRekening.add(barusakun); (Baris 39) Memasukkan objek rekening baru tersebut ke dalam ArrayList agar tersimpan aman dan tidak hilang atau tertimpa saat membuat akun lain.

- `akunAktif = barusakun;` (Baris 40) Mengatur agar rekening yang baru saja dibuat ini langsung menjadi akun yang sedang aktif untuk digunakan pada transaksi berikutnya.
- Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam `ArrayList` untuk dijadikan `akunAktif`

```

68@
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89

```

```

case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Belum ada rekening yang terdaftar di sistem.");
    } else {
        System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
        String cariNo = input.nextLine();
        boolean ditemukan = false;
        for (Rekening r : daftarRekening) {
            if (r.nomorRekening.equalsIgnoreCase(cariNo)) {
                akunAktif = r;
                ditemukan = true;
                System.out.println("Berhasil mengganti ke akun milik: " + r.namaPemilik);
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Error: Nomor rekening tidak ditemukan!");
        }
        break;
    }

```

- Baris 72-74 : Meminta pengguna memasukkan nomor rekening yang ingin dipilih, lalu menyimpannya di variabel
- Baris 76-83: Melakukan penelusuran (for-each) ke seluruh daftar rekening. Jika nomor rekening cocok (`equalsIgnoreCase`), sistem akan mengubah `akunAktif` ke rekening tersebut, menandai `ditemukan = true`, menampilkan pesan sukses, lalu menghentikan pencarian dengan `break`.
- Baris 86-87: Jika setelah dicari sampai selesai nomor rekening tidak ada (`!ditemukan`), sistem akan menampilkan pesan "Error: Nomor rekening tidak ditemukan!".

BAB III

KESIMPULAN

Praktikum Modul 1 ini memberikan pemahaman mendalam mengenai implementasi konsep pemrograman berorientasi objek (PBO) melalui kelas Rekening dan Main. Pembuatan kelas bertindak sebagai cetak biru (*blueprint*), sementara *constructor* mempermudah inisialisasi *state* awal seperti nomor rekening, nama pemilik, dan saldo. Logika transaksi yang dibangun pada *method* tarikTunai berhasil melakukan validasi keamanan secara akurat, mulai dari batas minimal penarikan Rp10.000 hingga pengecekan ketersediaan saldo.

Masalah utama pada program awal—di mana pembuatan akun baru selalu menimpa akun lama—berhasil diatasi melalui penerapan struktur data *Collection* berupa `ArrayList<Rekening>`. Dengan mengubah arsitektur penyimpanan ini pada `Main.java`, setiap objek rekening yang dibuat melalui menu "Buka Rekening Baru" akan tersimpan aman ke dalam daftar. Hal ini memungkinkan sistem untuk menampung banyak akun sekaligus secara dinamis tanpa kehilangan data dari nasabah sebelumnya.

Selain itu, keberadaan penelusuran (*looping*) dan pencocokan data pada fitur *switch account* (case 5) makin melengkapi sistem perbankan mini ini. Pengguna dapat dengan mudah mencari dan memilih nomor rekening yang tersimpan di dalam `ArrayList` untuk dijadikan sebagai akun aktif. Secara keseluruhan, kombinasi antara enkapsulasi logika transaksi dan manajemen data dinamis berbasis *Collection* berhasil menciptakan aplikasi perbankan yang fleksibel serta interaktif.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul Praktikum Pemrograman Berorientasi Objek: Class, Object, Constructor, dan Method*. Laboratorium Komputer, Departemen Informatika, Universitas Andalas.